

OpenAI Specification — Best Invest Properties

Version: 1.2

Date: 29 September 2026

API: OpenAI Responses API

Called through: Make

Publication: only after admin review

1. What OpenAI does on the platform

OpenAI works in **two separate stages**:

1. **Proposed inputs and assessments.** From the property data and the source materials it is given, OpenAI proposes figures and assessments for the admin to check.
2. **Analysis text.** After the admin has approved the values and the score, OpenAI writes the readable **Investment Analysis** text.

How it works:

1. Make sends OpenAI the property data and the source materials, each with its link and date (section 4.1), together with the fixed Stage 1 instructions (section 5.1).
2. OpenAI returns proposals in a fixed structure (section 6.1). Each proposal has its sources and assumptions. Where there is no source, the item is marked as a gap.
3. The platform checks the proposals automatically, and saves them as **"awaiting review"**.
4. The admin checks and corrects the proposals. Bubble then applies the fixed formulas for the yield and the score to the approved values.
5. Make sends OpenAI the approved figures, the score and the reviewed facts (section 4.2), together with the fixed Stage 2 instructions (section 5.2).
6. OpenAI returns the text in a fixed structure (section 6.2). The platform checks it automatically: every statement must point to a supplied fact, no new numbers may appear, and words like "guaranteed" or "risk-free" are not allowed.
7. The admin reviews the text and approves it. Only then do investors see it.

OpenAI does **not**:

- apply the formulas for acquisition cost, yield, points or the total score;
- invent a figure or a source — without a source, the item is marked as a gap;
- decide on its own what a "good investment" is;
- search the internet;
- see the investor's name, email, phone or any other investor data, or the contact details of developer representatives;
- publish anything, or have anything used, without admin approval;
- replace a financial, legal or investment adviser.

If OpenAI is unavailable, unchecked data is not published. Approved figures stay visible, and the property page still shows the score breakdown with the note "Narrative analysis is being reviewed".

2. What is in the MVP

Stage 1 — proposed inputs and assessments for the admin to check:

- the expected monthly rent;
- annual operating costs;
- the data needed for the total acquisition cost (purchase taxes and costs);
- qualitative assessments for the categories that need judgement: Rental Demand & Tenant Quality, Purchase Value & Market Position, Growth & Resale Potential, Risk & Investor Protection.

Every proposal is based only on the supplied source materials and has its sources and assumptions. The developer's claimed rent is never used as the proposed rent. These proposals appear on the Project Review screen next to their sources. The admin approves or corrects each one before it is used.

Stage 2 — the investment analysis text:

- a short neutral summary;
- 2–4 strengths, each linked to the facts it is based on;
- 1–4 risks or limitations;
- an explanation of the score, without changing it;
- a list of missing data;
- the standard disclaimer.

Not in the MVP:

- rewriting developer descriptions with AI;
- a chat assistant for investors.

3. Model

```
endpoint: POST /v1/responses
primary model: gpt-5.6-luna
fallback / manual high-quality regeneration: gpt-5.6-terra
store: false
tools: []
text.format: json_schema, strict: true
```

```
Stage 1 (proposals):    reasoning.effort: medium, max_output_tokens: 2500
Stage 2 (analysis text): reasoning.effort: low,    max_output_tokens: 1800
```

The primary model is the lower-cost one. Before launch both stages are tested on a set of example properties. If the quality is not good enough, the platform switches to the higher-quality model. The model name is a setting, so switching requires no development work.

4. Data contracts Bubble → OpenAI

Neither contract contains investor data or the contact details of developer representatives. The numbers, sources and links are examples.

4.1 Stage 1 — property data and source materials

```
{
  "schema_version": "1.0",
  "proposal_id": "prp_01J...",
  "unit_public_id": "unt_01J...",
  "locale": "en",
  "property": {
    "country": "Spain",
    "city": "Alicante",
    "area_name": "Example area",
    "property_type": "Apartment",
    "bedrooms": 2,
    "bathrooms": 2,
    "indoor_area_m2": 78,
    "features": ["pool", "parking", "gated_area"],
    "completion_date": "2027-06-30",
    "availability": "available"
  },
  "developer_information": [
    { "key": "dev.price", "label": "Purchase price", "value": 280000, "currency": "EUR" },
    { "key": "dev.claimed_monthly_rent", "label": "Developer's claimed rent", "value": 2100,
    "currency": "EUR" }
  ],
  "source_materials": [
    {
      "source_id": "src_01",
      "type": "rental_comparable",
      "title": "2-bed apartment, same district, pool and parking",
      "url": "https://example-source.test/listing/123",
      "retrieved_at": "2026-09-20",
      "content": { "monthly_rent": 1850, "bedrooms": 2, "indoor_area_m2": 80, "features": ["pool",
      "parking"] }
    },
    {
      "source_id": "src_02",
      "type": "country_costs",
      "title": "Approved purchase costs and running costs – Spain",
      "url": null,
      "retrieved_at": "2026-09-15",
      "content": {
        "purchase_costs": [
          { "label": "Purchase tax", "rate": 0.10 },
          { "label": "Notary and registry", "amount": 1500 }
        ],
        "running_costs": [
          { "label": "Community fees", "annual_amount": 1200 }
        ]
      }
    }
  ],
  "comparables_summary": {
    "count": 7,
```

```

    "rent_min": 1700,
    "rent_median": 1900,
    "rent_max": 2050,
    "note": "technical processing of the sample by Bubble, not an approved figure"
  },
  "requested_items": [
    "expected_monthly_rent",
    "annual_operating_costs",
    "purchase_costs",
    "assessment.demand",
    "assessment.value",
    "assessment.growth",
    "assessment.risk"
  ],
  "rating_scale": "set by the scoring methodology (to be agreed)"
}

```

What to notice:

- `source_materials` are the only allowed basis for proposals. Each one has a `source_id`, a link (where one exists) and a date;
- `developer_information` is the developer's claim and is never treated as independently verified;
- `comparables_summary` is calculated by Bubble and is given for context only;
- a requested item without supporting sources is returned as a gap, not guessed.

4.2 Stage 2 — approved figures and score

This is sent only after the admin has approved the Stage 1 values and Bubble has calculated the figures and score.

```

{
  "schema_version": "1.0",
  "analysis_id": "ana_01J...",
  "unit_public_id": "unt_01J...",
  "locale": "en",
  "property": {
    "country": "Spain",
    "city": "Alicante",
    "area_name": "Example area",
    "property_type": "Apartment",
    "bedrooms": 2,
    "bathrooms": 2,
    "indoor_area_m2": 78,
    "completion_date": "2027-06-30",
    "availability": "available"
  },
  "financials": {
    "currency": "EUR",
    "price": 280000,
    "expected_monthly_rent": 1900,
    "occupancy_rate": 0.9,
    "annual_gross_rent": 22800,

```

```

    "purchase_costs": 28000,
    "annual_net_income": 17248,
    "gross_yield": 0.0814,
    "net_yield": 0.056,
    "approved_at": "2026-09-22T09:30:00Z",
    "calculation_version": "fin-v3",
    "calculated_at": "2026-09-22T10:00:00Z"
  },
  "score": {
    "total": 77,
    "verdict": "strong",
    "model_version": "score-v5",
    "components": [
      { "key": "income", "label": "Rental Income & Net Yield", "points": 21, "max_points": 30,
"source_key": "financials.net_yield" },
      { "key": "demand", "label": "Rental Demand & Tenant Quality", "points": 16, "max_points": 20,
"source_key": "assessment.demand" },
      { "key": "value", "label": "Purchase Value & Market Position", "points": 16, "max_points": 20,
"source_key": "assessment.value" },
      { "key": "growth", "label": "Growth & Resale Potential", "points": 12, "max_points": 15,
"source_key": "assessment.growth" },
      {
        "key": "risk",
        "label": "Risk & Investor Protection",
        "scale_direction": "higher_points_mean_lower_risk",
        "points": 12,
        "max_points": 15,
        "source_key": "assessment.risk"
      }
    ]
  },
  "approved_facts": [
    {
      "key": "project.completion_date",
      "value": "2027-06-30",
      "label": "Expected completion",
      "source_type": "project_verified"
    }
  ],
  "analysis_facts": [
    {
      "key": "fact.rent_comparables",
      "tag": "source",
      "body": "Comparable long-let rentals in the same district, reviewed by the admin."
    },
    {
      "key": "fact.developer_inputs",
      "tag": "developer",
      "body": "Purchase price, unit size and completion date supplied by the developer and not
independently verified."
    },
    {
      "key": "fact.service_charge",
      "tag": "gap",

```

```

    "body": "No service charge schedule supplied for the building."
  }
],
"missing_data_keys": ["project.service_charge_schedule"],
"disclaimer_key": "investment_analysis_standard_v1"
}

```

What to notice:

- every figure in `financials` and `score` has been approved by the admin or calculated by Bubble from approved values;
- `score.components` contains exactly **five** items with keys `income`, `demand`, `value`, `growth`, `risk` and maximums 30/20/20/15/15;
- the `analysis_facts` array carries tags `source` / `developer` / `estimate` / `gap` — the model may cite them as sources and **must** mention every `gap` fact under risks or `missing_data`;
- for the `risk` category the payload includes `scale_direction` (higher points = lower risk), so the model does not invert it in the text.

Do not send, in either stage:

- investor identity/contact;
- contact details of developer representatives;
- developer verification documents;
- exact private address;
- admin private notes;
- unpublished competing units;
- Make/Bubble secrets;
- full legal text where a versioned disclaimer key is enough.

5. System/developer prompts

These instructions are fixed in the platform and cannot be changed by developers or investors. The property JSON is passed as `input_text` after the instructions. Do not mix the prompt and the data in one free-text block.

5.1 Stage 1 — proposals

Prompt version: `bip-proposals-v1`.

You prepare proposed inputs and assessments for Best Invest Properties. An admin reviews everything you return; nothing is published or used without approval.

Use only the property data and the `source_materials` in the supplied JSON. Do not use outside knowledge. Never invent figures, sources, market conditions, legal rules, taxes, demand or future appreciation.

For each requested item, propose a value only if the `source_materials` support it, and cite the `source_ids` you used. List your assumptions. If the sources do not support a value, return value null with status "gap" and add the item to `missing_data`.

`developer_information` is the developer's claim, never independently verified. Never use the developer's claimed rent as the expected rent.

Do not calculate acquisition cost, yield, net income, points or the total score; the platform does this. For the "Risk & Investor Protection" assessment a higher rating means LOWER assessed risk.

Return only JSON matching the supplied strict schema.

5.2 Stage 2 — analysis text

Prompt version: `bip-investment-analysis-v1`.

You write factual investment-property analysis for Best Invest Properties.

Use only facts present in the supplied JSON. Never calculate, infer, estimate, round, or replace financial values. Never invent market conditions, legal rules, taxes, neighbourhood claims, demand, future appreciation, guarantees, or advice.

The financial figures, score, verdict, and score components are approved inputs. Explain them; do not challenge or change them. There are exactly five score categories; never invent, merge, or omit one. For the "Risk & Investor Protection" category a higher number of points means LOWER assessed risk – never describe a high score in that category as high risk.

Every strength and risk must cite one or more `source_keys` that exist in the input. If evidence is missing, add the key to `missing_data` rather than guessing. Any `analysis_fact` tagged "gap" must appear either in risks or in `missing_data`. Facts tagged "developer" must be attributed to the developer and never presented as independently verified.

Use neutral, professional language. Avoid certainty about returns. Do not use "guaranteed", "safe", "best", "will increase", "risk-free", or equivalent claims. State clearly that projected and actual returns may differ, that all financial figures are estimates based on the displayed assumptions, and that the content is not financial, tax, or legal advice.

Return only JSON matching the supplied strict schema.

6. Structured Output schemas

OpenAI must answer in exactly these structures. Any answer in a different shape is rejected automatically.

6.1 Stage 1 — proposals

```
{
  "name": "proposed_inputs",
  "strict": true,
  "schema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "schema_version": { "type": "string", "enum": ["1.0"] },
```

```

"proposals": {
  "type": "array",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "item_key": {
        "type": "string",
        "enum": [
          "expected_monthly_rent",
          "annual_operating_costs",
          "purchase_costs",
          "assessment.demand",
          "assessment.value",
          "assessment.growth",
          "assessment.risk"
        ]
      },
      "status": { "type": "string", "enum": ["proposed", "gap"] },
      "value": { "type": ["number", "null"] },
      "unit": { "type": "string", "enum": ["EUR_per_month", "EUR_per_year", "EUR", "rating"] },
      "breakdown": {
        "type": "array",
        "items": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "label": { "type": "string" },
            "amount": { "type": "number" },
            "source_ids": { "type": "array", "items": { "type": "string" } }
          },
          "required": ["label", "amount", "source_ids"]
        }
      },
      "rationale": { "type": "string" },
      "assumptions": { "type": "array", "items": { "type": "string" } },
      "source_ids": { "type": "array", "items": { "type": "string" } }
    },
    "required": ["item_key", "status", "value", "unit", "breakdown", "rationale",
"assumptions", "source_ids"]
  },
  "missing_data": { "type": "array", "items": { "type": "string" } }
},
"required": ["schema_version", "proposals", "missing_data"]
}

```

After parsing, the platform checks that:

- every requested item appears exactly once;
- a `proposed` item has a value and at least one `source_id` that exists in the input;
- a `gap` item has `value: null` and appears in `missing_data`;
- the proposed rent is not simply the developer's claimed rent.

The proposals are then saved as **"awaiting review"** for the admin.

6.2 Stage 2 — analysis text

```
{
  "name": "investment_analysis",
  "strict": true,
  "schema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "schema_version": { "type": "string", "enum": ["1.0"] },
      "headline": { "type": "string" },
      "summary": { "type": "string" },
      "strengths": {
        "type": "array",
        "items": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "title": { "type": "string" },
            "commentary": { "type": "string" },
            "source_keys": {
              "type": "array",
              "items": { "type": "string" }
            }
          }
        },
        "required": ["title", "commentary", "source_keys"]
      },
      "risks": {
        "type": "array",
        "items": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "title": { "type": "string" },
            "commentary": { "type": "string" },
            "severity": { "type": "string", "enum": ["low", "medium", "high", "unknown"] },
            "source_keys": {
              "type": "array",
              "items": { "type": "string" }
            }
          },
          "required": ["title", "commentary", "severity", "source_keys"]
        },
        "score_explanation": { "type": "string" },
        "missing_data": {
          "type": "array",
          "items": { "type": "string" }
        },
        "disclaimer_key": { "type": "string", "enum": ["investment_analysis_standard_v1"] }
      },
    },
  },
}
```

```

    "required": [
      "schema_version",
      "headline",
      "summary",
      "strengths",
      "risks",
      "score_explanation",
      "missing_data",
      "disclaimer_key"
    ]
  }
}

```

After parsing, the platform checks text lengths, 2–4 strengths, 1–4 risks, at least one source key per item, and that no number appears that is not in the input.

Structured Outputs guarantees that the answer has the right structure, but **not** that it is factually correct. That is why the automatic checks and the admin review are always required. Any change to the number of score categories or to the input structure requires a new prompt version; the schema version changes only when the output structure changes.

7. Full request examples

7.1 Stage 1 — proposals

```

{
  "model": "gpt-5.6-luna",
  "store": false,
  "reasoning": { "effort": "medium" },
  "max_output_tokens": 2500,
  "metadata": {
    "use_case": "bip_proposed_inputs",
    "prompt_version": "bip-proposals-v1",
    "proposal_id": "prp_01J..."
  },
  "instructions": "<versioned Stage 1 prompt>",
  "input": [
    {
      "role": "user",
      "content": [
        { "type": "input_text", "text": "<property data and source materials JSON>" }
      ]
    }
  ],
  "text": {
    "format": { "type": "json_schema", "name": "proposed_inputs", "strict": true, "schema": {} }
  }
}

```

7.2 Stage 2 — analysis text

```
{
  "model": "gpt-5.6-luna",
  "store": false,
  "reasoning": { "effort": "low" },
  "max_output_tokens": 1800,
  "metadata": {
    "use_case": "bip_investment_analysis",
    "prompt_version": "bip-investment-analysis-v1",
    "analysis_id": "ana_01J..."
  },
  "instructions": "<versioned Stage 2 prompt>",
  "input": [
    {
      "role": "user",
      "content": [
        { "type": "input_text", "text": "<approved property snapshot JSON>" }
      ]
    }
  ],
  "text": {
    "format": { "type": "json_schema", "name": "investment_analysis", "strict": true, "schema": {} }
  }
}
```

The model is not given web search or any other tools, so it cannot bring in facts from outside the supplied data.

How OpenAI keeps the data. Under OpenAI's published data controls:

- `store: false` means the response is **not saved as application state** for later retrieval through the API. It does not mean that OpenAI keeps no record of the request.
- By default, API requests and responses may be kept in **abuse-monitoring logs for up to 30 days**, or longer where required by law or needed to protect OpenAI's services or others from harm.
- Data sent to the API is **not used to train OpenAI models** unless the organisation opts in.
- **Zero Data Retention** is available only to eligible customers approved by OpenAI. Whether the platform needs it is to be decided before launch.

8. Official OpenAI sources

- [Models and model selection](#)
- [GPT-5.6 Terra](#)
- [Responses API: create response](#)
- [Data controls in the OpenAI platform](#)
- [How your data is used to improve model performance](#)
- [Moderations API](#)
- [Evals API](#)